

МИНИСТЕРСТВО ОБРАЗОВАНИЯ КИРОВСКОЙ ОБЛАСТИ
Кировское областное государственное профессиональное образовательное
бюджетное учреждение
«Слободской колледж педагогики и социальных отношений»

ОТЧЕТ

по производственной практике

ПМ.02. Осуществление интеграции программных модулей

Студент

Пестов Арсений Николаевич

Группа 23П-1

Специальность 09.02.07

Информационные системы и
программирование

Руководитель практики от колледжа:

Седов Алексей Сергеевич

Руководитель практики от организации:

_____ *Зязин Сергей Валерьевич*

подпись

УТВЕРЖДАЮ:

Начальник

Зязин Сергей Валерьевич

Наименование организации

Управление социального развития
администрации Слободского района

_____ / _____

подпись

расшифровка

М. П.

2025-2026 уч. год

ОГЛАВЛЕНИЕ

1. Характеристика объекта практики (юридический адрес, специализация, структура)
2. Описание рабочего места
3. Состав программного и технического обеспечения, имеющегося на предприятии, их назначение
4. Описание выполненных видов работ
 - 4.1. Проанализировать предметную область
 - 4.2. Спроектировать ERD, UML (Use case)
 - 4.3. Спроектировать на основе предметной области базу данных
 - 4.4. Произвести импорт данных в БД
 - 4.5. Разработать программный модуль на основе предметной области
 - 4.6. Произвести интеграцию с API
 - 4.7. Произвести отладку программных модулей
 - 4.8. Доработать программные модули обработкой исключительных ситуаций
 - 4.9. Рефакторинг программного кода
 - 4.10. Провести ручное тестирование
 - 4.11. Провести модульное тестирование
 - 4.12. Провести тестирование интеграции
 - 4.13. Фиксация результатов в системе контроля версий
 - 4.14. Разработка руководства оператора
 - 4.15. Разработка пояснительной записки
5. Заключение

1. Характеристика объекта практики

Объектом практики является отдел социального развития администрации города Слободского.

Юридический адрес: ул. Советская, 86, Слободской, Кировская обл., 613150.

Специализация отдела: организация социальной поддержки граждан, назначение пособий, работа с льготными категориями, ведение базы получателей социальных услуг.

Структура отдела: руководитель отдела, несколько специалистов по направлениям (социальные выплаты, работа с пожилыми, поддержка семей с детьми), бухгалтерия, системный администратор (один человек на весь отдел). Всего около 10–15 сотрудников.

2. Описание рабочего места

Практика проходила на моём личном ноутбуке. На нём были установлены все необходимые для прохождения практики программы, включая офисный пакет, среды разработки, программы тестирования и прочие вспомогательные утилиты.

3. Состав программного и технического обеспечения, имеющегося на предприятии, их назначение

В отделе социального развития используется стабильное ПО:

- **Операционная система:** Windows 10 Pro (лицензионная).
- **1С:Предприятие** – для учёта выплат и отчётности.
- **Антивирус ClamAV** – бесплатный, сканирует по требованию.
- **Дополнительное ПО:** Total Commander (файловый менеджер), IrfanView (просмотр изображений), Firefox (браузер).
- **Техническое обеспечение:** компьютеры (Intel Core i3/i5, 8 ГБ ОЗУ), принтеры, локальная сеть через коммутатор.

4. Описание выполненных видов работ

4.1. Анализ предметной области

Предметной областью является процесс приёма, регистрации, обработки и контроля исполнения обращений граждан в администрации муниципального образования. Обращения могут поступать через различные каналы: личный приём, электронную почту, веб-портал.

Основные бизнес-процессы включают:

- подачу обращения гражданином с указанием темы, текста и (опционально) прикреплённых файлов;
- проверку и регистрацию обращения оператором приёмной, классификацию по категориям;
- назначение ответственного исполнителя из структурного подразделения;
- исполнение обращения, изменение его статуса, комментирование;
- контроль сроков и формирование отчётности для руководства.

Выделены следующие категории пользователей: граждане (заявители), операторы приёмной, исполнители (сотрудники отделов) и администратор системы.

На основе анализа построена диаграмма вариантов использования (Use Case), отображающая роли и их функциональные возможности (см. Рисунок 1).



Рисунок 1 – Диаграмма вариантов использования

4.2. Проектирование ERD и UML (Use Case)

Диаграмма вариантов использования детализирует взаимодействие акторов с системой. Гражданин регистрируется, подаёт обращения, просматривает свои заявки и прикрепляет файлы. Оператор обрабатывает обращения, назначает исполнителя и изменяет статус. Исполнитель работает с назначенными обращениями, добавляет комментарии. Администратор управляет пользователями и формирует отчёты.

Логическая модель данных представлена в виде ER-диаграммы (см. Рисунок 2), которая отображает сущности и их взаимосвязи.

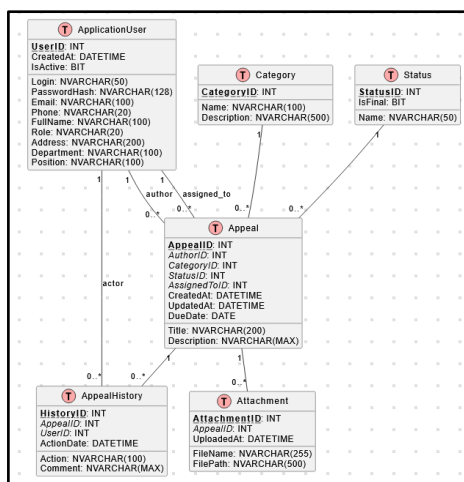


Рисунок 2 – ER-диаграмма базы данных

4.3. Проектирование базы данных

На основе ER-модели разработана реляционная база данных, нормализованная до третьей нормальной формы. Определены следующие таблицы:

- **ApplicationUser** – пользователи приложения (логин, пароль, роль, ФИО и др.);
- **Appeal** – обращения (автор, заголовок, описание, категория, статус, исполнитель);
- **Category** – категории обращений;
- **Status** – статусы обращений (с признаком конечности);

- **AppealHistory** – история действий (создание, смена статуса, комментарии);
- **Attachment** – прикрепленные файлы.

Связи между таблицами реализованы через внешние ключи с каскадным удалением для истории и вложений.

Физическая модель реализована в СУБД Microsoft SQL Server. Скрипты создания таблиц, ограничений и индексов подготовлены в виде отдельных файлов.

4.4. Импорт данных в БД

Для заполнения базы тестовыми данными разработан T-SQL скрипт, генерирующий более 2600 записей. Скрипт добавляет справочные данные (категории, статусы), 300 граждан, операторов, исполнителей, администратора, а также около 900 обращений со случайным распределением статусов и исполнителей. Для каждого обращения создаётся запись в истории, а для 10% обращений – прикрепленные файлы.

Выполнение скрипта в среде SQL Server Management Studio продемонстрировано на Рисунке 3.

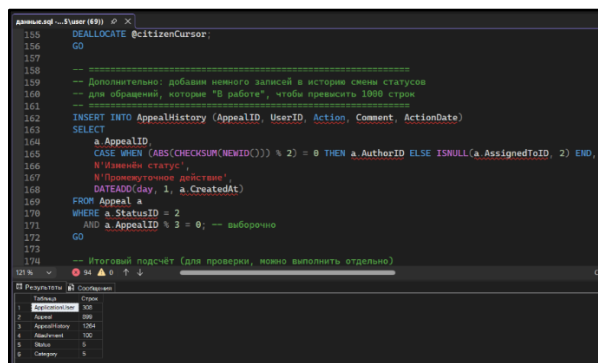


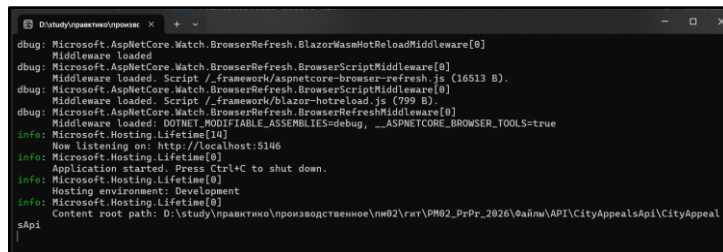
Рисунок 3 – Импорт тестовых данных

4.5. Разработка программного модуля

Программный комплекс состоит из двух частей: серверного Web API и клиентского настольного приложения.

Серверная часть реализована на [ASP.NET Core 10](#). Предоставляет REST-эндпоинты для аутентификации, управления обращениями, пользователями,

категориями, статусами и вложениями. Доступ к данным осуществляется через Entity Framework Core. Аутентификация выполняется по паре логин/пароль, передаваемых в заголовках запросов. Консоль запущенного сервера представлена на рисунке 4.



```

D:\study\практика\проект x  +  -
dbug: Microsoft.AspNetCore.Middleware.BrowserRefresh.BlazorWasHotReloadMiddleware[0]
      Middleware loaded
dbug: Microsoft.AspNetCore.Middleware.BrowserRefresh.BrowserScriptMiddleware[0]
      Middleware loaded. Script /_framework/aspnetcore-browser-refresh.js (16513 B).
dbug: Microsoft.AspNetCore.Middleware.BrowserRefresh.BrowserScriptMiddleware[0]
      Middleware loaded. Script /_framework/blazor-hotreload.js (799 B).
dbug: Microsoft.AspNetCore.Middleware.BrowserRefresh.BrowserRefreshMiddleware[0]
      Middleware loaded. DOTNET_MODIFIABLE_ASSEMBLIES=debug, __ASPNETCORE_BROWSER_TOOLS=true
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5146
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: D:\study\практика\производственное\пм02\run\PM02_PrPr_2026\@айм\API\CityAppealsApi\CityAppeal
sApi

```

Рисунок 4 – Командная строка API ASP.NET

Клиентская часть – приложение WPF (.NET 10) с архитектурой MVVM. Пользовательский интерфейс (показан на рисунке 5) адаптируется под роль авторизованного пользователя. Взаимодействие с сервером инкапсулировано в сервисе ApiService, который выполняет HTTP-запросы и десериализует ответы.

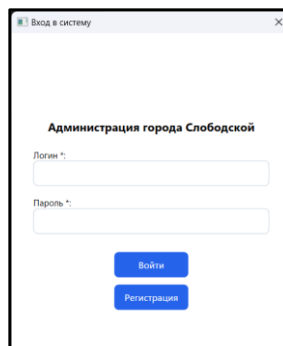


Рисунок 5 – Окно входа клиентского приложения

4.6. Интеграция с API

Интеграция выполнена путём замены прямого доступа к базе данных в клиентском приложении на вызовы API. Сервис ApiService формирует HTTP-запросы, добавляет авторизационные заголовки X-Login и X-Password, обрабатывает ответы. При ошибках сервера или сети пользователю отображаются соответствующие сообщения.

Пример интеграции – окно деталей обращения (Рисунок 6), данные в которое загружаются посредством GET-запроса к `/api/appeals/{id}`.

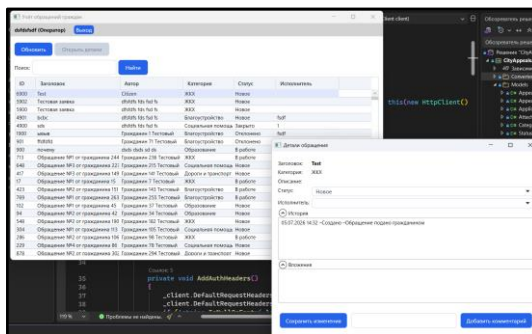


Рисунок 6 – Отображение данных, полученных через API

4.7. Отладка программных модулей

Отладка проводилась с использованием встроенных средств Visual Studio, а также инструментов тестирования HTTP-запросов (Postman). Для серверной части проверялась корректность маршрутов и ответов, для клиентской – правильность формирования запросов и обработки ответов. Выявленные ошибки, связанные с циклическими ссылками при сериализации JSON, были устранены путём проецирования данных на анонимные типы.

4.8. Доработка обработкой исключительных ситуаций

В клиентском приложении реализована централизованная обработка исключений в классе ApiService. В зависимости от кода состояния HTTP и наличия сообщения от сервера формируется понятное пользователю сообщение. Например, ошибка 401 «Unauthorized» преобразуется в «Неверный логин или пароль, либо недостаточно прав», ошибка 404 – «Запрашиваемый объект не найден». При сетевых сбоях выводится «Ошибка соединения».

4.9. Рефакторинг программного кода

В ходе работы выполнен ряд улучшений:

- Изменена архитектура клиента: добавлен слой сервиса ApiService, избавив ViewModel-классы от прямого использования ApplicationDbContext.
- Все команды, связанные с вводом-выводом, переведены на асинхронное выполнение с помощью AsyncRelayCommand, что исключает блокировку пользовательского интерфейса.
- Модели данных адаптированы под JSON-сериализацию.

- Проведена очистка неиспользуемых зависимостей и унификация обработки ответов от API.

4.10. Ручное тестирование

Ручное тестирование проводилось по заранее составленным тест-кейсам, охватывающим функциональность всех ролей. Всего разработано и выполнено 29 тестов (16 тест-кейсов для клиентского приложения и 13 запросов в Postman для тестирования API). В таблице 1 приведён фрагмент тест-кейсов (полный перечень – в репозитории).

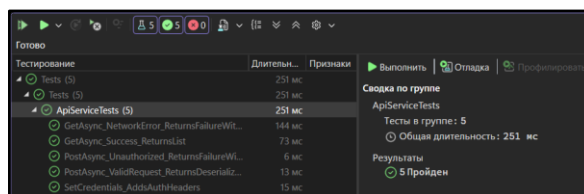
Таблица 1 – Фрагмент ручных тест-кейсов

Поле	Значение
ИД	ТС-01
Название теста	Вход с корректными данными
Приоритет	Высокий
Предусловия	Запущено приложение, пользователь зарегистрирован
Шаги воспроизведения	1. В поле «Логин» ввести operator1 2. В поле «Пароль» ввести 1 3. Нажать «Войти»
Ожидаемый результат	Открывается главное окно с панелью «Оператор»
Фактический результат	Соответствует
Статус	Пройден

Все тесты завершились успешно, что подтверждает соответствие реализованной функциональности требованиям.

4.11. Модульное тестирование

Модульное тестирование проведено для клиентского сервиса ApiService с использованием фреймворков NUnit и Moq. Были протестированы сценарии успешного выполнения запросов, ошибок аутентификации, сетевых исключений и проверки добавления заголовков. Результаты тестирования показаны на Рисунке 7.



Тестирование	Длительн...	Признаки
Tests (5)	251 мс	
Tests (5)	251 мс	
ApiServiceTests (5)	251 мс	
GetAsync_NetworkError_ReturnsFailureWith...	144 мс	
GetAsync_Success_ReturnsList	73 мс	
PostAsync_Unauthorized_ReturnsFailureWith...	6 мс	
PostAsync_ValidRequest_ReturnsDeserializ...	13 мс	
SetCredentials_AddsAuthHeaders	15 мс	

Сводка по группе

ApiServiceTests

Тесты в группе: 5

Общая длительность: 251 мс

Результаты

5 Пройден

Рисунок 7 – Результаты модульного тестирования

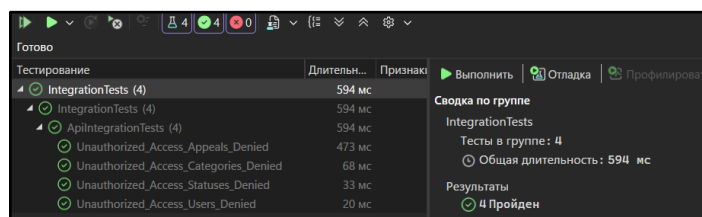
Листинг 1 демонстрирует пример теста.

```
[Test]
public async Task Unauthorized_Access_Statuses_Denied()
{
    ClearAuthHeaders();
    var response = await _client.GetAsync("/api/statuses");
    Assert.That(response.StatusCode, Is.EqualTo(HttpStatusCode.Unauthorized));
}
```

Листинг 1 – Модульный тест обработки ошибки 401

4.12. Интеграционное тестирование

Интеграционные тесты выполнены для API с помощью WebApplicationFactory и in-memory базы данных. Сценарии включают полный жизненный цикл обращения (регистрация, вход, создание, получение списка, удаление), работу оператора по обновлению статуса и назначению исполнителя, доступ администратора к списку пользователей и проверку отказа в доступе без авторизации. Все тесты прошли успешно (Рисунок 8).



Тестирование	Длительн...	Признаки
IntegrationTests (4)	594 мс	
IntegrationTests (4)	594 мс	
ApiIntegrationTests (4)	594 мс	
Unauthorized_Access_Appeals_Denied	473 мс	
Unauthorized_Access_Categories_Denied	68 мс	
Unauthorized_Access_Statuses_Denied	33 мс	
Unauthorized_Access_Users_Denied	20 мс	

Сводка по группе

IntegrationTests

Тесты в группе: 4

Общая длительность: 594 мс

Результаты

4 Пройден

Рисунок 8 – Результаты интеграционного тестирования

4.13. Фиксация результатов в системе контроля версий

Исходный код проекта размещён в системе контроля версий Git. В процессе работы регулярно выполнялись коммиты с осмысленными сообщениями, фиксирующие ключевые изменения: создание структуры проекта,

реализацию API, перевод клиента на API, исправление ошибок, добавление тестов и документации. Репозиторий обеспечивает сохранность истории разработки и возможность отката к предыдущим состояниям. Репозиторий доступен публично - https://github.com/ScioSilentium13/PM02_PrPr_2026.

4.14. Разработка руководства оператора

Разработан документ «Руководство оператора», описывающий порядок работы с приложением для всех категорий пользователей. Документ содержит инструкции по запуску, входу в систему, основным операциям (создание, обработка, удаление обращений, администрирование пользователей) и действиям в нештатных ситуациях. Основное содержание руководства приведено в репозитории.

4.15. Разработка пояснительной записки

Пояснительная записка содержит описание архитектуры системы, перечень реализованных функций, обоснование технических решений, результаты тестирования и инструкции по развёртыванию. Документ позволяет получить полное представление о программном комплексе, его структуре и логике работы. Полный текст записки представлен в репозитории.

5. Заключение

В ходе выполнения практики по модулю «Осуществление интеграции программных модулей» была достигнута поставленная цель – разработана информационная система учёта обращений граждан, состоящая из серверного API и клиентского приложения.

Решены следующие задачи:

- Проведён анализ предметной области, построены UML-диаграммы (Use Case) и ER-модель.
- Спроектирована нормализованная база данных и выполнено её развёртывание с заполнением тестовыми данными.
- Разработан программный модуль в виде Web API на [ASP.NET Core](#) и WPF-клиента.

- Реализована интеграция клиента с API, включая авторизацию и обработку ошибок.
- Выполнена отладка, рефакторинг, ручное, модульное и интеграционное тестирование.
- Подготовлены руководство оператора и пояснительная записка.

Полученные навыки проектирования, разработки и тестирования многоуровневых приложений, а также работы с системой контроля версий соответствуют задачам практической подготовки. Разработанное решение готово к внедрению в опытную эксплуатацию.